Università di Genova

Embedded Systems

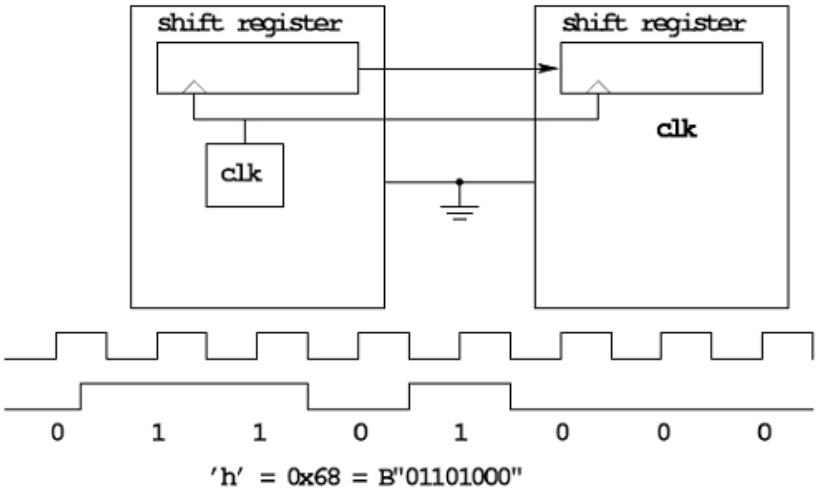
SPI

Prof. Enrico Simetti

SPI

Synchronous serial communication:

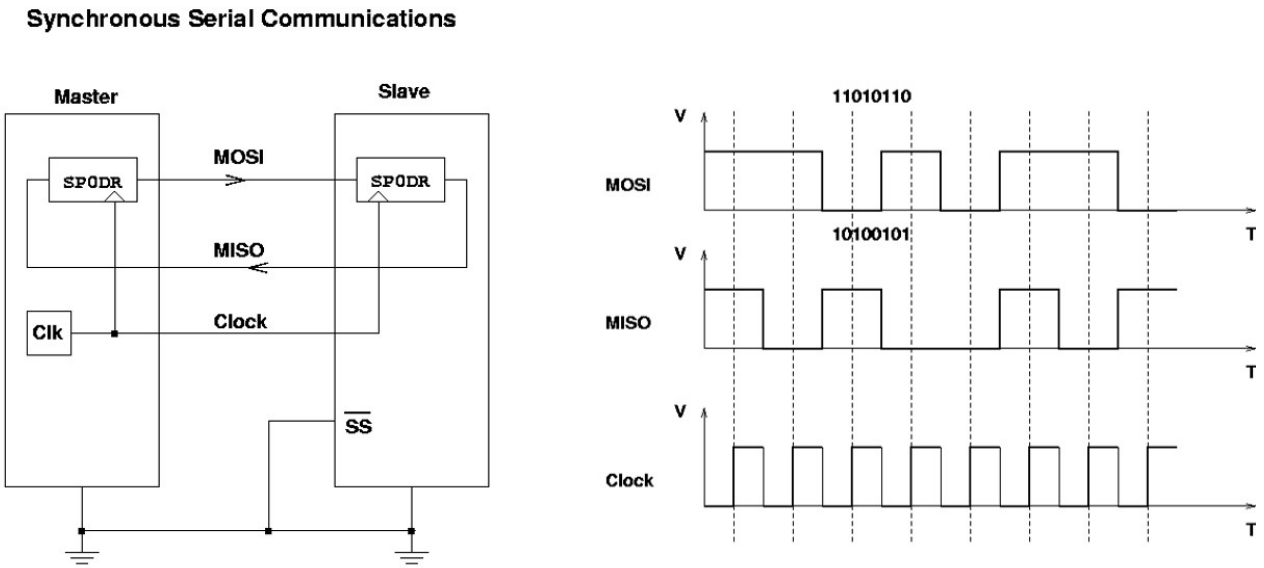
- the transmitter puts the data byte it wants to send into an internal shift register
- the transmitter uses a clock to shift the data bits out of its shift register onto an external data pin
- the receiver uses the input clock from the transmitter to sample incoming data
- the receiver puts the data on the input pin on an internal shift register.



SPI Communication

Full-duplex communication:

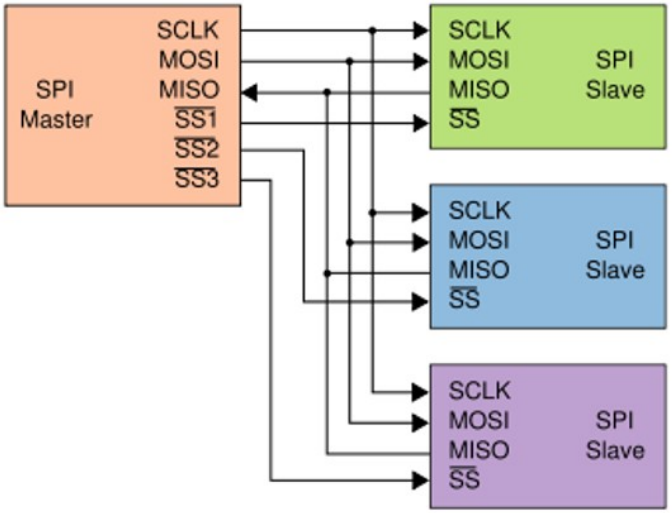
- two data lines are used: MISO (Master In Slave Out) and MOSI (Master Out Slave In)
- the master generates a clock signal and send it to the slave
- both master and slave transmit data at the same time
- after transmission, the contents of the two shift registers are swapped.

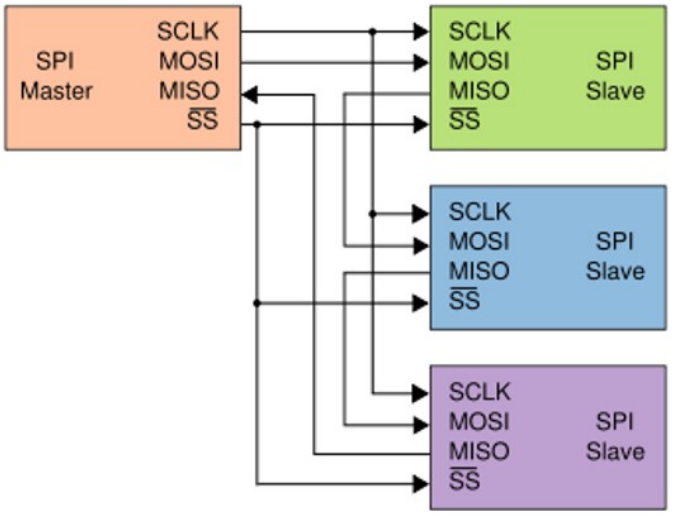


Multi-node Configurations

Multi-node communication:

- a master can talk to many different slaves (a slave select signal may be used for guaranteeing that only one slave is enabled to write on the MISO line)
- alternatively, nodes can be connected in a daisy-chain topology





Comparison over RS232 (UART)

Main advantages over RS232

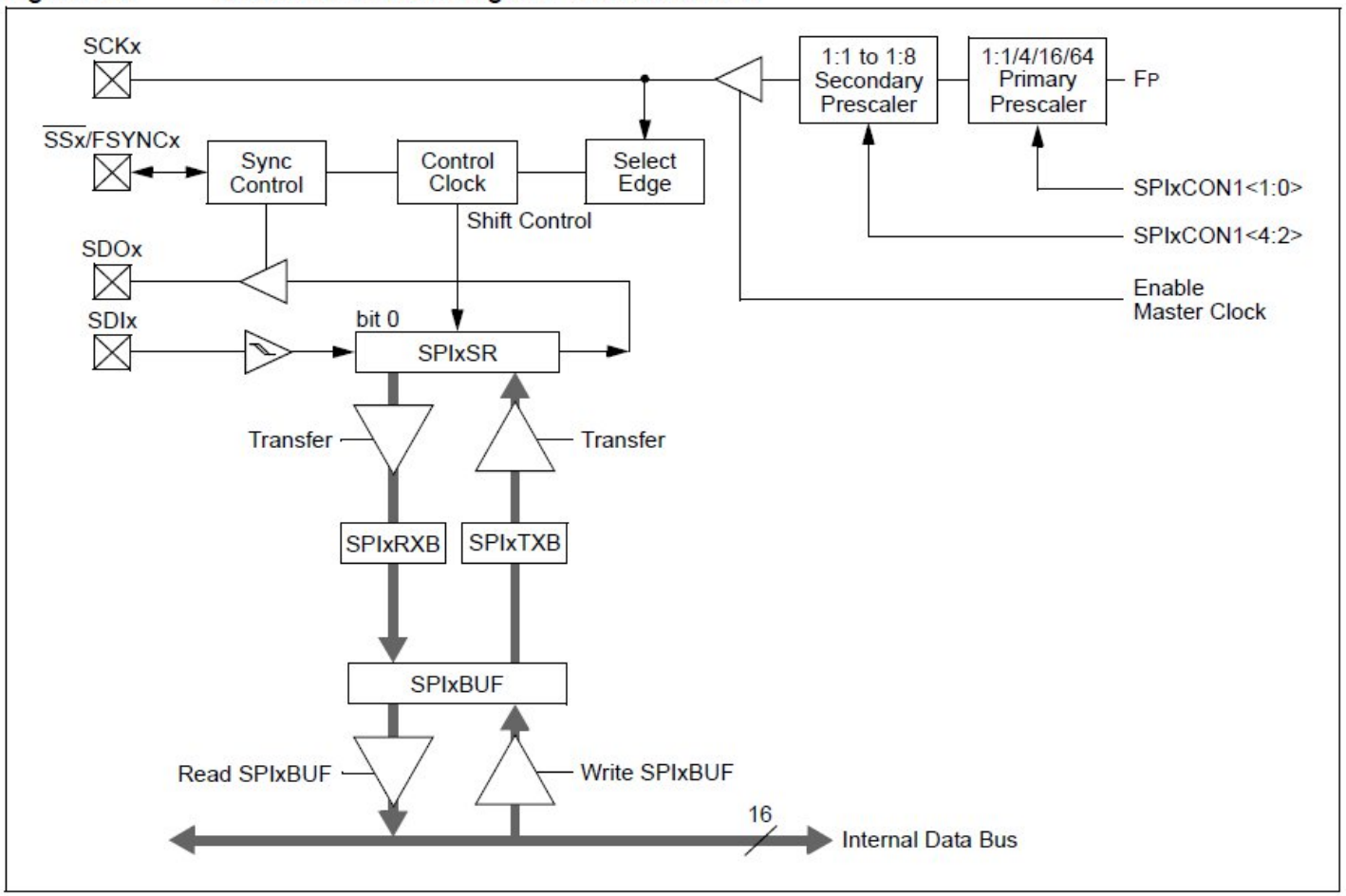
- High throughput (up to 70 Mbps)
- High bandwidth
- Transmission not limited to 8-bit words
- Slaves use the master's clock, and don't need precision oscillators
- Multi-node communication available

Main disadvantages over RS232

- More expensive for clock generation circuitry
- More pins and wires required
- Supports only one master device at a given time
- Less robust to noise
- No flow control or parity check
- Only handles short distances

dsPIC33F SPI Module

Figure 1-1: SPIx Module Block Diagram – Standard Mode



SPIxBUF Register

The SPIxTXB and the SPIxRXB are unidirectional, 16-bit registers that share the SFR address of the SPIxBUF register. If the user application writes data to be transmitted to the SPIxBUF register, internally the data is written to the SPIxTXB register. Similarly, when the user application reads the received data from the SPIxBUF register, internally the data is read from the SPIxRXB register.

SPIxSTAT Register

Register 2-1: SPIxSTAT: SPIx Status and Control Register

R/W-0	U-0	R/W-0	U-0	U-0	R/W-0	R/W-0	R/W-0
SPIEN	—	SPISIDL	—	—	SPIBEC2(1)	SPIBEC1(1)	SPIBEC0(1)
bit 15							bit 8

R/W-0	R/C-0, HS	R/W-0	R/W-0	R/W-0	R/W-0	R-0, HS, HC	R-0, HS, HC
SRMPT	SPIROV	SRXMPT	SISEL2(1)	SISEL1(1)	SISEL0(1)	SPITBF	SPIRBF
bit 7							bit 0

SPIxSTAT: SPI Status registers, contains the flags that notify the program of certain events.

- **SPIEN** needed to enable the SPI peripheral.
- **SPITBF** tells if the transmit buffer is full
- **SPIRBF** tells if the receiver buffer is full
- **SPIROV** tells if a receiver buffer overrun has occurred

SPIxCON1 Register

Register 2-2: SPIxCON1: SPIx Control Register 1

U-0	U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
—	—	—	DISSCK(5)	DISSDO	MODE16	SMP(2)	CKE(1)
bit 15							bit 8

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
SSEN(4)	CKP	MSTEN	SPRE2(3)	SPRE1(3)	SPRE0(3)	PPRE1(3)	PPRE0(3)
bit 7							bit 0

SPIxCON: SPI Control register.

- MSTEN: enables SPI master mode
- MODE16: selects 16/8 bits mode
- PPRE: primary prescaler
- SPRE: secondary prescaler
- CKP specifies the idle value of the clock

SPI prescalers are different from timer prescalers!

SPIxBUF Register

SPIxBUF: it is the register that contains the byte/word to be sent to the SPI slave

SPI Overflow

If a new data word has been shifted into the SPIxSR register and the previous SPIxBUF register contents have not been read, the SPIROV bit (inside SPIxSTAT) is set. Any received data in the SPIxSR register is not transferred, and further data reception is disabled until the SPIROV bit is cleared. The SPIROV bit is not cleared automatically by the module; it must be cleared by the user application.

SPI Example

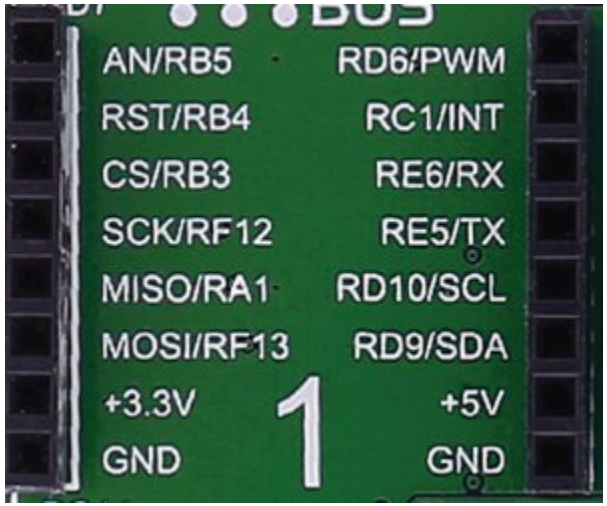
Example: writing one character to the SPI (Fcy = 2 MHz, SPI clock 400 KHz)

```
SPI1CONbits.MSTEN = 1;  // master mode
SPI1CONbits.MODE16 = 0; // 8-bit mode
SPI1CONbits.PPRE = 3;   // 1:1 primary prescaler
SPI1CONbits.SPRE = 3;   // 5:1 secondary prescaler
SPI1STATbits.SPIEN = 1; // enable SPI

while(SPI1STATbits.SPITBF == 1); // wait until the tx buffer is not full
SPI1BUF = 'x';                   // send the 'x' character
while(SPI1STATbits.SPIRBF == 0); // wait until data has arrived
data = SPI1BUF;
while(SPI1STATbits.SPITBF == 1); // wait until the tx buffer is not full
SPI1BUF = 22;                    // send the byte containing the value 22
while(SPI1STATbits.SPIRBF == 0); // wait until data has arrived
data = SPI1BUF;
```

IMU Clicker Board (SPI)





SPI Remapping

SCK, SDO, SDI must be remapped accordingly.

The SDI label on the addon board is the input for the chip, hence is connected to the MOSI/SDO (output) of the micro. Viceversa for SDO and MISO.

Chip selectors:

- CS1 = accelerometers
- CS2 = gyroscope
- CS3 = magnetometer

Only ONE chip selector can be ``active'' at a given time!

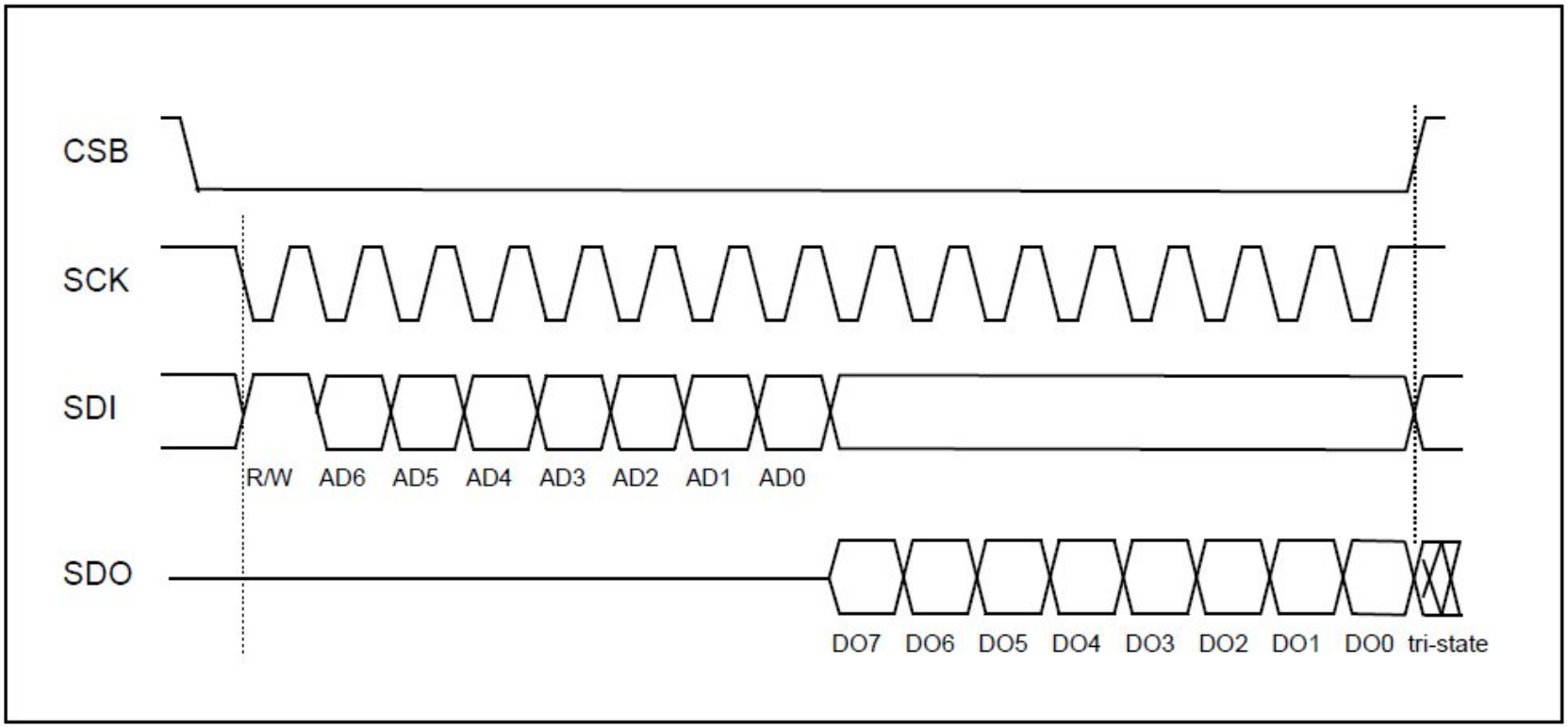
SPI Remapping

Remapping configuration:

```
TRISAbits.TRISA1 = 1;  // RA1-RPI17  MISO
TRISFbits.TRISF12 = 0; // RF12-RP108 SCK
TRISFbits.TRISF13 = 0; // RF13-RP109 MOSI

RPINR20bits.SDI1R = 0b0010001; // MISO (SDI1) - RPI17
RPOR12bits.RP109R = 0b000101;  // MOSI (SDO1) - RF13;
RPOR11bits.RP108R = 0b000110;  // SCK1;
```

SPI Time Diagrams



SPI Operations

The idle value for the clock is 1. Please set the CKP bit accordingly.

A single byte exchange is performed by setting the MSB (R/W) to 1 for a read, to 0 for a write operation. The other bits inside the byte sent to the IMU represent the address.

In a read operation, the MCU must send 0x00 to enable the generation of the clock to get the answer, after sending the address.

In a write operation, the MCU must send the value to be written after the address.

SPI IMU Example

Example reading register 0x40 (Chip ID):

```
MAG_CS = 0; // change to the PORT connected to the chip select
read_addr = 0x40;
while (SPI1STATbits.SPITBF == 1);
SPI1BUF = read_addr | 0x80; // setting the MSB to 1
while (SPI1STATbits.SPIRBF == 0);
trash = SPI1BUF; // read to prevent buffer overrun
while (SPI1STATbits.SPITBF == 1);
SPI1BUF = 0x00; // clocking out zeros so that the other chip can send the value
while (SPI1STATbits.SPIRBF == 0);
value_from_chip = SPI1BUF; // get the value from the register
MAG_CS = 1;
```

the returned value should be 0x32.

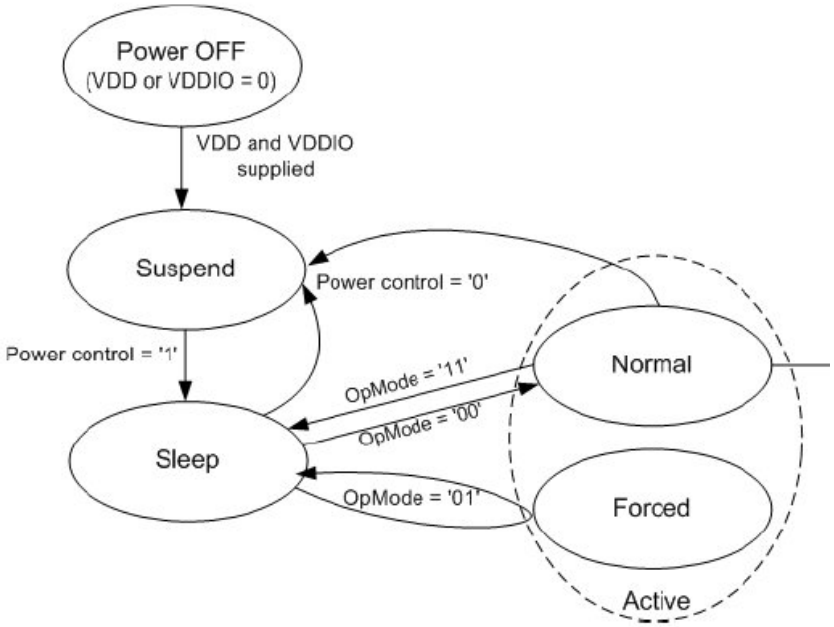
Remember to have the Accelerometer and Gyroscope CS disabled (=1)!

Multi-byte Communication

	Control byte								Data byte								Data byte								Data byte								
Start	RW	Register adress (02h)								Data register - adress 02h								Data register - adress 03h								Data register - adress 04h								Stop
CSB = 0	1	0	0	0	0	0	1	0		X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	CSB = 1

Multiple read operations are possible by keeping CSB low and continuing the data transfer. Only the first register address has to be written. Addresses are automatically incremented after each read access as long as CSB stays low.

IMU Magnetometer State Machine



A state machine regulates the functioning of the magnetometer.

Magnetometer Configuration

At the beginning, the device is in Suspended mode. To pass in Sleep mode, the register 0x4B must have the ``power bit'' set to 1. All the other bits can be zero, hence you can write 0x01 to the register.

(0x4B) Bit	Name	Description
Bit 7	Soft Reset '1'	One of the soft reset trigger bits.
Bit 6	-	(fixed to 0)
Bit 5	-	(fixed to 0)
Bit 4	-	(fixed to 0)
Bit 3	-	(fixed to 0)
Bit 2	SPI3en	Enable bit for SPI3 mode
Bit 1	Soft Reset '1'	One of the soft reset trigger bits.
Bit 0	Power Control bit	When set to "0", suspend mode is selected

To pass in Suspended mode, the device requires up to 3 ms.

Magnetometer Configuration

Then, the device can change from Sleep to Active if the OpMode bits of the register 0x4C are set to 0b00 (normal mode). All the other bits can be zero, hence you can write 0x00 to the register.

(0x4C) Bit	Name	Description
Bit 7	Adv. ST <1>	Advanced self-test control bit 1
Bit 6	Adv. ST <0>	Advanced self-test control bit 0
Bit 5	Data rate <2>	Data rate control bit 2
Bit 4	Data rate <1>	Data rate control bit 1
Bit 3	Data rate <0>	Data rate control bit 0
Bit 2	Opmode <1>	Operation mode control bit 1
Bit 1	Opmode <0>	Operation mode control bit 0
Bit 0	Self Test	Normal self-test control bit

Three “Data rate” bits control the output data rate (ODR) of the magnetometer part:

Table 51: Output data rate (ODR) setting *(0x4C)*

(0x4C) Data rate <2:0>	Magnetometer output data rate (ODR) [Hz]
000b	10 (default)
001b	2
010b	6
011b	8
100b	15
101b	20
110b	25
111b	30

After the command to go in sleep mode, remember to wait for 3 ms before commanding the system to go into active mode

Magnetometer Configuration

(0x4C) Opmode <1:0>	Magnetometer operation mode[9]
00b	Normal mode
01b	Forced mode
10b	Reserved, do not use
11b	Sleep Mode

Magnetometer Data Registers

Magnetic data is contained in multiple registers. For example, the x axis is a 13 bit data split in registers 0x42 and 0x43. Data is represented in two's complement

Table 40: LSB part of x-axis magnetic field, register *(0x42)*

(0x42) Bit	Name	Description
Bit 7	DATAX_lsb <4>	Bit 4 of x-axis magnetic field data
Bit 6	DATAX_lsb <3>	Bit 3 of x-axis magnetic field data
Bit 5	DATAX_lsb <2>	Bit 2 of x-axis magnetic field data
Bit 4	DATAX_lsb <1>	Bit 1 of x-axis magnetic field data
Bit 3	DATAX_lsb <0>	Bit 0 of x-axis magnetic field data = x LSB
Bit 2	-	(fixed to 0)
Bit 1	-	(fixed to 0)
Bit 0	SelfTestX	Self-test result flag for x-axis, default is "1"

MAG Register *(0x43)*

Register *(0x43)* contains the MSB part of x-axis magnetic field data.

Table 41: MSB part of x-axis magnetic field, register *(0x43)*

(0x43) Bit	Name	Description
Bit 7	DATAX_msb <12>	Bit 12 of x-axis magnetic field data = x MSB
Bit 6	DATAX_msb <11>	Bit 11 of x-axis magnetic field data
Bit 5	DATAX_msb <10>	Bit 10 of x-axis magnetic field data
Bit 4	DATAX_msb <9>	Bit 9 of x-axis magnetic field data
Bit 3	DATAX_msb <8>	Bit 8 of x-axis magnetic field data
Bit 2	DATAX_msb <7>	Bit 7 of x-axis magnetic field data
Bit 1	DATAX_msb <6>	Bit 6 of x-axis magnetic field data
Bit 0	DATAX_msb <5>	Bit 5 of x-axis magnetic field data

Reading Magnetometer's Data

To correctly read, one has to read both registers (you can send 0x42, followed by two 0x00 and you'll get both registers).

Then, they have to be combined together, and the result must be divided to put in the correct scale.

LSB and MSB Manipulation

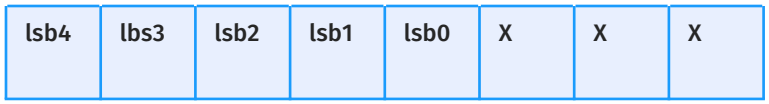
How to put LSB and MSB together in a two-byte signed integer:

- read the LSB and MSB parts using SPI;
- clear the three least significant bits of the LSB by ANDing with a mask (a specific constant that has 1 everywhere, except for the three least significant bits)
- left-shift by 8 the MSB and then do the OR with the previously found masked LSB value
- divide by 8 (or right-shift*) to get in the correct scale.

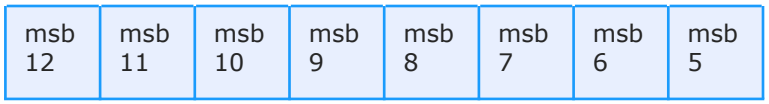
**right-shift must be arithmetic and not logical – compiler dependent*

Manipulating Bytes

Mag 0x42



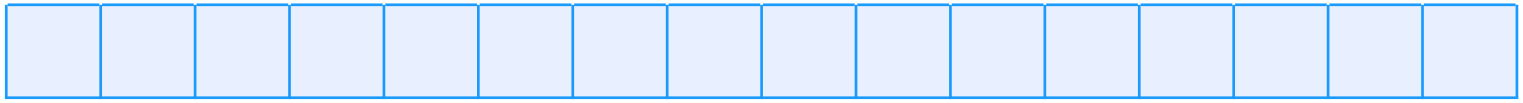
Mag 0x43



msb



lsb



Manipulating Bytes

Mag 0x42

lsb4	**lbs3**	**lsb2**	**lsb1**	**lsb0**	**X**	**X**	**X**

Mag 0x43

msb 12	msb 11	msb 10	msb 9	msb 8	msb 7	msb 6	msb 5

Read mag register 0x42 into the variable lsb

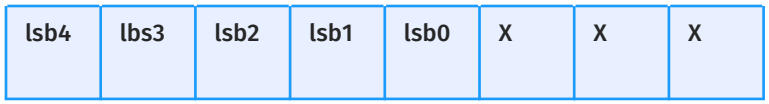
msb

lsb

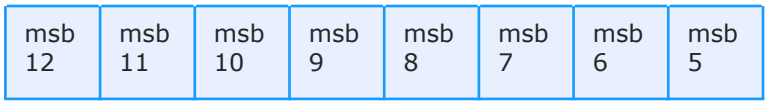
0	0	0	0	0	0	0	0	lsb4	lbs3	lsb2	lsb1	lsb0	X	X	X

Manipulating Bytes

Mag 0x42



Mag 0x43

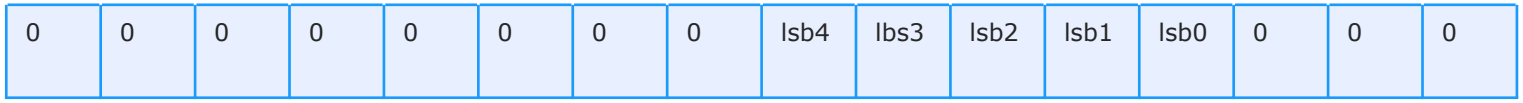


Lsb & 0x00F8

msb



lsb



Manipulating Bytes

Mag 0x42

lsb4	**lbs3**	**lsb2**	**lsb1**	**lsb0**	**X**	**X**	**X**

Mag 0x43

msb 12	msb 11	msb 10	msb 9	msb 8	msb 7	msb 6	msb 5

Read mag register 0x43 into the variable msb

msb

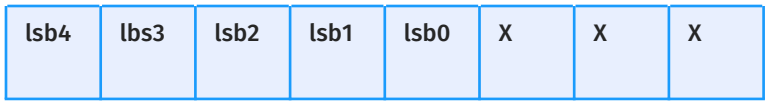
0	0	0	0	0	0	0	0	msb 12	msb 11	msb 10	msb 9	msb 8	msb 7	msb 6	msb 5

lsb

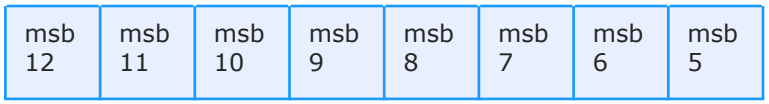
0	0	0	0	0	0	0	0	lsb4	lbs3	lsb2	lsb1	lsb0	0	0	0

Manipulating Bytes

Mag 0x42

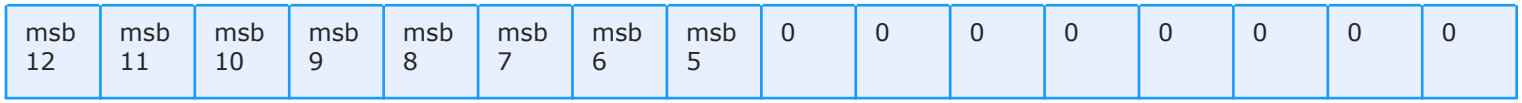


Mag 0x43

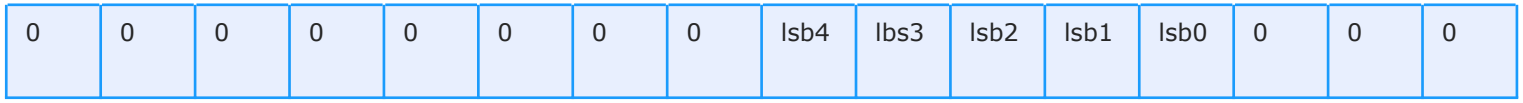


Shift variable msb by 8

msb



lsb



Manipulating Bytes

Mag 0x42

lsb4	lbs3	lsb2	lsb1	lsb0	X	X	X

Mag 0x43

msb 12	msb 11	msb 10	msb 9	msb 8	msb 7	msb 6	msb 5

result = mbs | lsb

msb

msb 12	msb 11	msb 10	msb 9	msb 8	msb 7	msb 6	msb 5	0	0	0	0	0	0	0	0

lsb

0	0	0	0	0	0	0	0	lsb4	lbs3	lsb2	lsb1	lsb0	0	0	0

result

msb 12	msb 11	msb 10	msb 9	msb 8	msb 7	msb 6	msb 5	lsb4	lbs3	lsb2	lsb1	lsb0	0	0	0

Manipulating Bytes

Mag 0x42

lsb4	lbs3	lsb2	lsb1	lsb0	X	X	X

Mag 0x43

msb 12	msb 11	msb 10	msb 9	msb 8	msb 7	msb 6	msb 5

result = result >> 3

msb

msb 12	msb 11	msb 10	msb 9	msb 8	msb 7	msb 6	msb 5	0	0	0	0	0	0	0	0

lsb

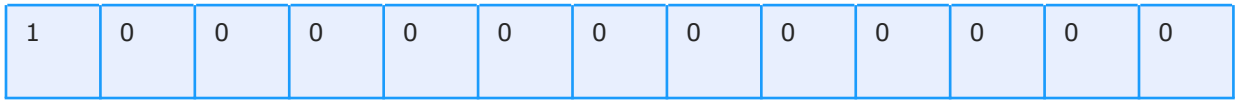
0	0	0	0	0	0	0	0	lsb4	lbs3	lsb2	lsb1	lsb0	0	0	0

result

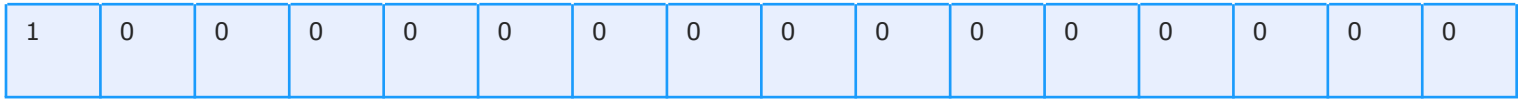
X	X	X	msb 12	msb 11	msb 10	msb 9	msb 8	msb 7	msb 6	msb 5	lsb4	lbs3	lsb2	lsb1	lsb0

When >> is an Arithmetic Shift

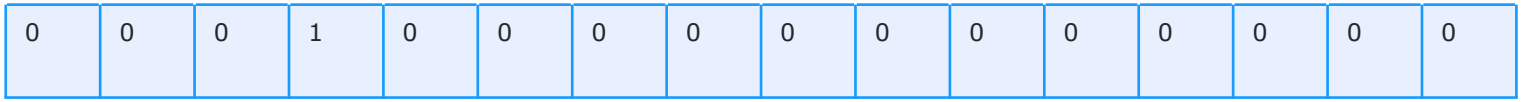
Original 13-bit data (-4096)



unsigned int result



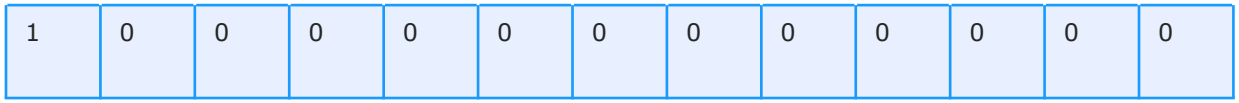
result >> 3



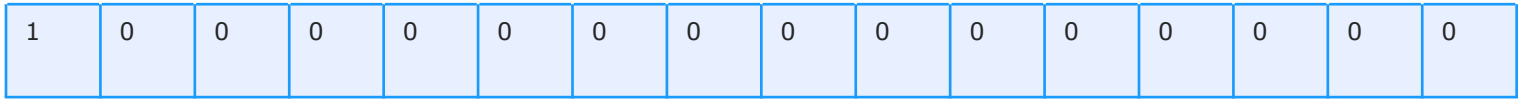
-4096 becomes 4096

When >> is an Arithmetic Shift

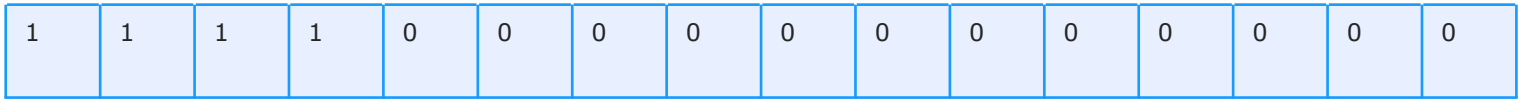
Original 13-bit data (-4096)



int result



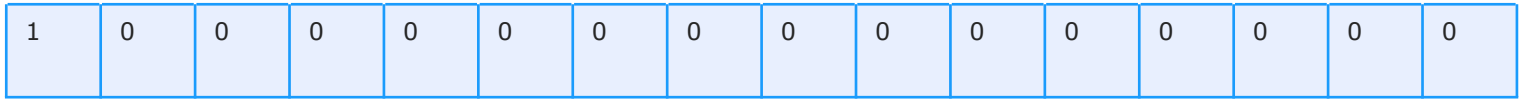
result >> 3



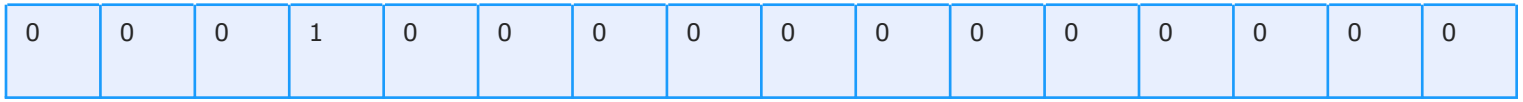
-4096 remains -4096 (in a 16-bit two-complement representation)

When >> is a Logical Shift

int result



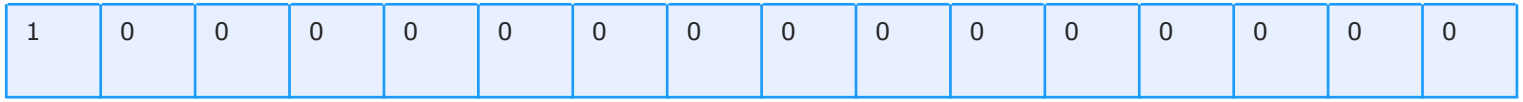
result >> 3



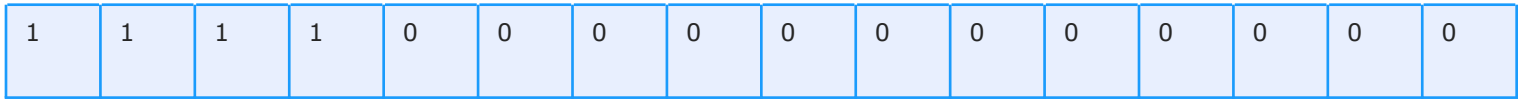
For signed types, the behavior is **implementation-defined** if the number is negative, but most modern CPUs use **arithmetic right shift**.

Division

int result



result / 8



Unsigned-Signed Assignments

unsigned int x = spi_read();
int y = x;

C++11(ISO/IEC 14882:2011) §4.7 Integral conversions

If the destination type is unsigned, the resulting value is the least unsigned integer congruent to the source integer (modulo 2^n where n is the number of bits used to represent the unsigned type). In a two's complement representation, this conversion is conceptual and there is no change in the bit pattern (if there is no truncation).

If the destination type is signed, the value is unchanged if it can be represented in the destination type (and bit-field width); otherwise, the value is implementation-defined.

- **Before C++20**, converting an out-of-range unsigned value to signed was **implementation-defined** because different binary representations were theoretically possible.
- **In practice**, nearly all modern systems use **two's complement**, making this a non-issue.
- **C++20 mandates two's complement**, ensuring this behavior is now **fully defined**.
- memcpy can be used to avoid any portability issues

Assignment 1

Exercise 1: create a function

```
unsigned int spi_write(unsigned int data);
```

that writes the data to the SPI, and returns the value read from the other device. Then use the function to solve the following exercise:

- Configure SPI;
- Make the magnetometer switch to Sleep mode; then make it go to active mode;
- Acquire the Chip ID and send it to the UART.

Assignment 2

Exercise 2:

- Configure SPI;
- Make the magnetometer switch to Sleep mode; then make it go to active mode;
- Acquire the x-axis of the magnetometer and send it to the UART at 10 Hz using the protocol $MAGX=xxx*, where xxx is the value.



Università
di Genova